

Complete AI/ML Interview Preparation Guide - 20+ LPA Package

Machine Learning (ML) - Core Questions

1. What is the difference between supervised, unsupervised, and reinforcement learning?

Answer:

- **Supervised Learning:** Uses labeled data to learn a mapping from inputs to outputs. Examples: Classification (spam detection), Regression (house price prediction)
- **Unsupervised Learning:** Finds patterns in data without labels. Examples: Clustering (customer segmentation), Dimensionality reduction (PCA)
- **Reinforcement Learning:** Agent learns through interaction with environment via rewards/penalties. Examples: Game playing, robotics

2. Explain bias-variance tradeoff

Answer:

- **Bias:** Error from oversimplifying the model (underfitting). High bias = model too simple
- **Variance:** Error from sensitivity to small fluctuations in training data (overfitting). High variance = model too complex
- **Tradeoff:** Need to balance both. Total error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$
- **Solutions:** Cross-validation, regularization, ensemble methods

3. What are the assumptions of linear regression?

Answer:

1. **Linearity:** Relationship between X and Y is linear
2. **Independence:** Observations are independent
3. **Homoscedasticity:** Constant variance of residuals
4. **Normality:** Residuals are normally distributed
5. **No multicollinearity:** Features aren't highly correlated

4. Explain different evaluation metrics for classification

Answer:

- **Accuracy:** $(TP+TN)/(TP+TN+FP+FN)$ - Good for balanced datasets
- **Precision:** $TP/(TP+FP)$ - Important when false positives are costly
- **Recall/Sensitivity:** $TP/(TP+FN)$ - Important when false negatives are costly

- **F1-Score:** $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$ - Harmonic mean
- **AUC-ROC:** Area under ROC curve - Good for binary classification
- **Specificity:** $\text{TN} / (\text{TN} + \text{FP})$ - True negative rate

5. What is cross-validation and its types?

Answer: Cross-validation estimates model performance on unseen data:

- **K-Fold CV:** Split data into k folds, train on k-1, validate on 1
- **Stratified K-Fold:** Maintains class distribution in each fold
- **Leave-One-Out (LOOCV):** Use 1 sample for validation, rest for training
- **Time Series CV:** Respects temporal order in time series data
- **Monte Carlo CV:** Random sampling for train/validation splits

Deep Learning (DL) - Advanced Questions

6. Explain backpropagation algorithm

Answer: Backpropagation computes gradients for neural network weights:

1. **Forward Pass:** Compute predictions and loss
2. **Backward Pass:** Compute gradients using chain rule
3. **Weight Update:** Update weights using gradient descent

Key equations:

- $\delta(l) = \nabla a(l) \odot \sigma'(z(l))$ (output layer)
- $\delta(l) = ((w(l+1))^T \delta(l+1)) \odot \sigma'(z(l))$ (hidden layers)
- $\partial C / \partial w = a(l-1) \delta(l)$

7. What are different activation functions and their pros/cons?

Answer:

- **ReLU:** $f(x) = \max(0, x)$. Fast, solves vanishing gradient. Cons: Dead neurons
- **Leaky ReLU:** $f(x) = \max(\alpha x, x)$. Prevents dead neurons
- **ELU:** Smooth, negative values. Better than ReLU for deep networks
- **Sigmoid:** $f(x) = 1 / (1 + e^{-x})$. Outputs 0-1. Cons: Vanishing gradient
- **Tanh:** $f(x) = \tanh(x)$. Outputs -1 to 1. Better than sigmoid
- **Swish:** $f(x) = x * \text{sigmoid}(x)$. Self-gated, smooth
- **GELU:** Gaussian Error Linear Unit. Used in transformers

8. Explain different types of neural networks

Answer:

- **Feedforward NN:** Basic architecture, no cycles
- **CNN:** Convolutional layers for image processing. Uses convolution, pooling
- **RNN:** Recurrent connections for sequential data. Suffers from vanishing gradient
- **LSTM:** Solves RNN's vanishing gradient with gates (forget, input, output)
- **GRU:** Simplified LSTM with reset and update gates
- **Transformer:** Self-attention mechanism, parallel processing
- **Autoencoder:** Encoder-decoder for dimensionality reduction/reconstruction
- **GAN:** Generator vs Discriminator for data generation

9. What is vanishing/exploding gradient problem?

Answer:

- **Vanishing Gradient:** Gradients become very small in deep networks, especially with sigmoid/tanh
- **Exploding Gradient:** Gradients become very large, causing unstable training
- **Solutions:**
 - Better activation functions (ReLU)
 - Proper weight initialization (Xavier, He)
 - Batch normalization
 - Gradient clipping (for exploding)
 - Skip connections (ResNet)
 - LSTM/GRU for RNNs

10. Explain batch normalization

Answer: Batch normalization normalizes inputs to each layer:

- **Process:** $\mu = \text{mean}(\text{batch})$, $\sigma^2 = \text{var}(\text{batch})$, $\text{BN}(x) = \gamma(x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$
- **Benefits:** Faster training, higher learning rates, regularization effect
- **Alternatives:** Layer norm, instance norm, group norm

Natural Language Processing (NLP)

11. Explain different word embeddings

Answer:

- **One-hot encoding:** Sparse vectors, no semantic meaning

- **Word2Vec:**
 - CBOW: Predicts word from context
 - Skip-gram: Predicts context from word
- **GloVe:** Global vectors, combines global statistics with local context
- **FastText:** Handles OOV words using subword information
- **BERT embeddings:** Contextual embeddings, bidirectional
- **Sentence embeddings:** Doc2Vec, Universal Sentence Encoder

12. What is attention mechanism?

Answer: Attention allows models to focus on relevant parts of input:

- **Formula:** $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$
- **Types:**
 - Self-attention: Q, K, V from same sequence
 - Cross-attention: Q from target, K, V from source
 - Multi-head: Multiple attention heads in parallel
- **Benefits:** Handles long sequences, interpretable, parallel computation

13. Explain transformer architecture

Answer: Transformer uses self-attention mechanism:

- **Encoder:** Multi-head self-attention + feed-forward (6 layers)
- **Decoder:** Masked self-attention + encoder-decoder attention + feed-forward
- **Key components:**
 - Positional encoding for sequence order
 - Layer normalization
 - Residual connections
- **Advantages:** Parallelizable, handles long dependencies

14. What are different NLP tasks and approaches?

Answer:

- **Text Classification:** Sentiment analysis, spam detection (BERT, RoBERTa)
- **Named Entity Recognition:** Identify entities (BiLSTM-CRF, BERT)
- **Question Answering:** Extract answers from text (BERT, T5)
- **Machine Translation:** Seq2seq, Transformer (T5, mT5)
- **Text Summarization:** Extractive/Abstractive (PEGASUS, BART)

- **Language Modeling:** Next word prediction (GPT series)

15. Explain BERT and its variants

Answer: BERT (Bidirectional Encoder Representations from Transformers):

- **Training:** Masked Language Modeling + Next Sentence Prediction
- **Architecture:** Transformer encoder
- **Variants:**
 - RoBERTa: Removes NSP, better training
 - ALBERT: Parameter sharing, factorized embeddings
 - DistilBERT: Smaller, faster version
 - DeBERTa: Disentangled attention

MLOps - Production & Operations

16. What is MLOps and why is it important?

Answer: MLOps combines ML, DevOps, and Data Engineering:

- **Purpose:** Automate ML lifecycle from development to production
- **Components:**
 - Version control (data, code, models)
 - CI/CD pipelines
 - Model monitoring
 - Infrastructure management
- **Benefits:** Faster deployment, better collaboration, model reliability

17. Explain model versioning and experiment tracking

Answer:

- **Model Versioning:**
 - Track model artifacts, hyperparameters, metrics
 - Tools: MLflow, Weights & Biases, DVC
- **Experiment Tracking:**
 - Log parameters, metrics, artifacts
 - Compare experiments
 - Reproduce results
- **Best Practices:** Semantic versioning, automated logging, metadata management

18. What is model drift and how to detect it?

Answer:

- **Data Drift:** Input distribution changes over time
- **Concept Drift:** Relationship between input-output changes
- **Detection Methods:**
 - Statistical tests (KS test, PSI)
 - Distance metrics (KL divergence)
 - Model performance monitoring
- **Solutions:** Retraining, adaptive models, ensemble methods

19. Explain different deployment strategies

Answer:

- **Batch Prediction:** Process data in batches, suitable for non-real-time
- **Online Prediction:** Real-time API endpoints
- **Edge Deployment:** Deploy on edge devices
- **A/B Testing:** Compare model performance
- **Blue-Green Deployment:** Switch between environments
- **Canary Deployment:** Gradual rollout
- **Shadow Deployment:** Run alongside existing model

20. What are containerization and orchestration in ML?

Answer:

- **Containerization (Docker):**
 - Package model with dependencies
 - Consistent environments
 - Easy deployment
- **Orchestration (Kubernetes):**
 - Manage containers at scale
 - Auto-scaling, load balancing
 - Service discovery
- **ML-specific tools:** Kubeflow, MLflow, Seldon

Generative AI

21. Explain different types of generative models

Answer:

- **Variational Autoencoders (VAE):**
 - Encoder-decoder with latent space
 - Probabilistic approach
- **Generative Adversarial Networks (GAN):**
 - Generator vs Discriminator
 - Adversarial training
- **Autoregressive Models:**
 - Generate sequence step by step (GPT)
- **Diffusion Models:**
 - Denoise process (DALL-E 2, Stable Diffusion)
- **Flow-based Models:**
 - Invertible transformations

22. What are Large Language Models (LLMs)?

Answer: Large Language Models are transformer-based models trained on massive text data:

- **Examples:** GPT-3/4, PaLM, LLaMA, Claude
- **Capabilities:**
 - Text generation
 - Few-shot learning
 - Code generation
 - Reasoning
- **Training:** Unsupervised pre-training + supervised fine-tuning + RLHF
- **Challenges:** Computational cost, hallucinations, bias

23. Explain prompt engineering techniques

Answer:

- **Zero-shot:** Direct instruction without examples
- **Few-shot:** Provide examples in prompt
- **Chain-of-Thought:** Step-by-step reasoning
- **Tree of Thoughts:** Explore multiple reasoning paths
- **Role-based:** Assign specific role to model

- **Template-based:** Structured prompt formats
- **Best Practices:** Clear instructions, examples, output format specification

24. What is Retrieval Augmented Generation (RAG)?

Answer: RAG combines retrieval with generation:

- **Process:**
 1. Retrieve relevant documents
 2. Use as context for generation
 3. Generate response based on retrieved info
- **Components:**
 - Document store (vector database)
 - Retriever (dense retrieval, BM25)
 - Generator (LLM)
- **Benefits:** Reduces hallucinations, up-to-date information, explainable

25. Explain fine-tuning techniques for LLMs

Answer:

- **Full Fine-tuning:** Update all parameters (expensive)
- **Parameter-Efficient Fine-tuning:**
 - LoRA: Low-rank adaptation
 - Adapters: Small modules between layers
 - Prefix tuning: Optimize only prefix tokens
- **Instruction Tuning:** Train on instruction-following data
- **RLHF:** Reinforcement Learning from Human Feedback
- **In-context Learning:** Learn from examples in prompt

Agentic AI

26. What is Agentic AI?

Answer: Agentic AI refers to AI systems that can act autonomously:

- **Characteristics:**
 - Goal-directed behavior
 - Decision-making capability
 - Environment interaction
 - Learning and adaptation

- **Components:**
 - Planning and reasoning
 - Tool use and API calls
 - Memory and context management
 - Multi-step task execution

27. Explain different agent architectures

Answer:

- **ReAct (Reasoning + Acting):**
 - Alternates between reasoning and action
 - Uses thought-action-observation loop
- **Plan-and-Execute:**
 - High-level planning then execution
 - Hierarchical approach
- **Multi-agent Systems:**
 - Multiple specialized agents
 - Coordination mechanisms
- **Tool-using Agents:**
 - Access to external tools/APIs
 - Function calling capabilities

28. What are the challenges in building autonomous agents?

Answer:

- **Reliability:** Ensuring consistent performance
- **Safety:** Preventing harmful actions
- **Hallucination:** Generating false information
- **Context Management:** Maintaining long-term memory
- **Tool Integration:** Seamless API usage
- **Error Handling:** Recovery from failures
- **Evaluation:** Measuring agent performance

29. Explain agent evaluation frameworks

Answer:

- **Task-specific Metrics:** Success rate, completion time

- **General Capabilities:**
 - Planning quality
 - Tool usage accuracy
 - Reasoning consistency
- **Safety Measures:**
 - Harmful action detection
 - Robustness testing
- **Frameworks:** AgentBench, ToolBench, HELM

30. What is the future of Agentic AI?

Answer:

- **Autonomous Assistants:** Personal and professional helpers
 - **Multi-modal Agents:** Text, vision, audio integration
 - **Collaborative Agents:** Human-AI collaboration
 - **Specialized Domains:** Healthcare, finance, education
 - **Challenges:** Ethics, regulation, job displacement
-

System Design & Architecture Questions

31. Design a recommendation system for e-commerce

Answer:

- **Data Collection:** User interactions, product features, demographics
- **Algorithms:** Collaborative filtering, content-based, hybrid
- **Architecture:**
 - Data pipeline (Kafka, Spark)
 - Feature store
 - Model serving (real-time/batch)
 - A/B testing framework
- **Metrics:** Precision@K, Recall@K, CTR, conversion rate
- **Challenges:** Cold start, scalability, diversity

32. How would you deploy a real-time ML model?

Answer:

- **Model Serving:** REST API, gRPC

- **Infrastructure:** Kubernetes, Docker
- **Load Balancing:** Handle traffic spikes
- **Monitoring:** Latency, throughput, accuracy
- **Caching:** Redis for frequent predictions
- **Auto-scaling:** Based on traffic patterns
- **Fallback:** Default predictions for failures

Coding & Practical Questions

33. Implement gradient descent from scratch

Answer:

```
python

import numpy as np

def gradient_descent(X, y, learning_rate=0.01, epochs=1000):
    m, n = X.shape
    theta = np.zeros(n)

    for i in range(epochs):
        # Forward pass
        predictions = X.dot(theta)
        cost = np.sum((predictions - y) ** 2) / (2 * m)

        # Backward pass
        gradient = X.T.dot(predictions - y) / m

        # Update parameters
        theta -= learning_rate * gradient

        if i % 100 == 0:
            print(f"Cost after {i} iterations: {cost}")

    return theta
```

34. Implement a simple neural network

Answer:

```
python
```

```

import numpy as np

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.W1 = np.random.randn(input_size, hidden_size) * 0.1
        self.b1 = np.zeros((1, hidden_size))
        self.W2 = np.random.randn(hidden_size, output_size) * 0.1
        self.b2 = np.zeros((1, output_size))

    def forward(self, X):
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = self.sigmoid(self.z2)
        return self.a2

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-np.clip(z, -250, 250)))

    def backward(self, X, y, output):
        m = X.shape[0]

        # Output layer gradients
        dz2 = output - y
        dW2 = np.dot(self.a1.T, dz2) / m
        db2 = np.sum(dz2, axis=0, keepdims=True) / m

        # Hidden layer gradients
        dz1 = np.dot(dz2, self.W2.T) * self.a1 * (1 - self.a1)
        dW1 = np.dot(X.T, dz1) / m
        db1 = np.sum(dz1, axis=0, keepdims=True) / m

        return dW1, db1, dW2, db2

```

Behavioral & Situational Questions

35. How do you handle model performance degradation in production?

Answer:

1. **Detection:** Set up monitoring alerts for key metrics
2. **Investigation:** Analyze data drift, concept drift, feature importance changes
3. **Root Cause:** Check data quality, infrastructure issues, external factors
4. **Immediate Action:** Rollback to previous model if critical

5. **Long-term Fix:** Retrain with recent data, feature engineering, architecture changes
6. **Prevention:** Continuous monitoring, automated retraining pipelines

36. Describe a challenging ML project you worked on

Answer Structure:

- **Problem:** Clear problem statement
- **Challenges:** Data quality, model selection, deployment issues
- **Approach:** Methodology, experiments, iterations
- **Results:** Quantitative improvements, business impact
- **Learnings:** Technical and business insights

Salary Negotiation Tips for 20+ LPA

1. **Highlight Impact:** Quantify your contributions (improved accuracy by X%, reduced costs by Y%)
2. **Technical Depth:** Demonstrate deep understanding of end-to-end ML pipeline
3. **Business Acumen:** Show how ML drives business value
4. **Leadership:** Mention mentoring, cross-functional collaboration
5. **Continuous Learning:** Stay updated with latest technologies
6. **Portfolio:** Maintain strong GitHub with diverse projects
7. **Certifications:** AWS ML, Google Cloud ML, Azure AI certifications
8. **Publications:** Technical blogs, conference talks, research papers

Companies Offering 20+ LPA for AI/ML Roles

- **Product Companies:** Google, Microsoft, Amazon, Meta, Apple
- **Indian Tech:** Flipkart, Ola, Swiggy, Zomato, Paytm
- **Startups:** Razorpay, CRED, Meesho, Udaan
- **Consulting:** McKinsey, BCG, Deloitte
- **Finance:** Goldman Sachs, JP Morgan, American Express
- **AI Companies:** OpenAI, Anthropic, Hugging Face

Final Preparation Strategy

1. **Technical Foundation:** Master fundamentals deeply
2. **Hands-on Practice:** Implement algorithms from scratch
3. **System Design:** Practice ML system architecture
4. **Mock Interviews:** Practice with peers or platforms

5. **Portfolio Projects:** Build end-to-end ML applications
6. **Stay Current:** Follow latest research and industry trends
7. **Communication:** Practice explaining complex concepts simply